Azure End-to-End Data Engineering Project (AdventureWorks Analytics)

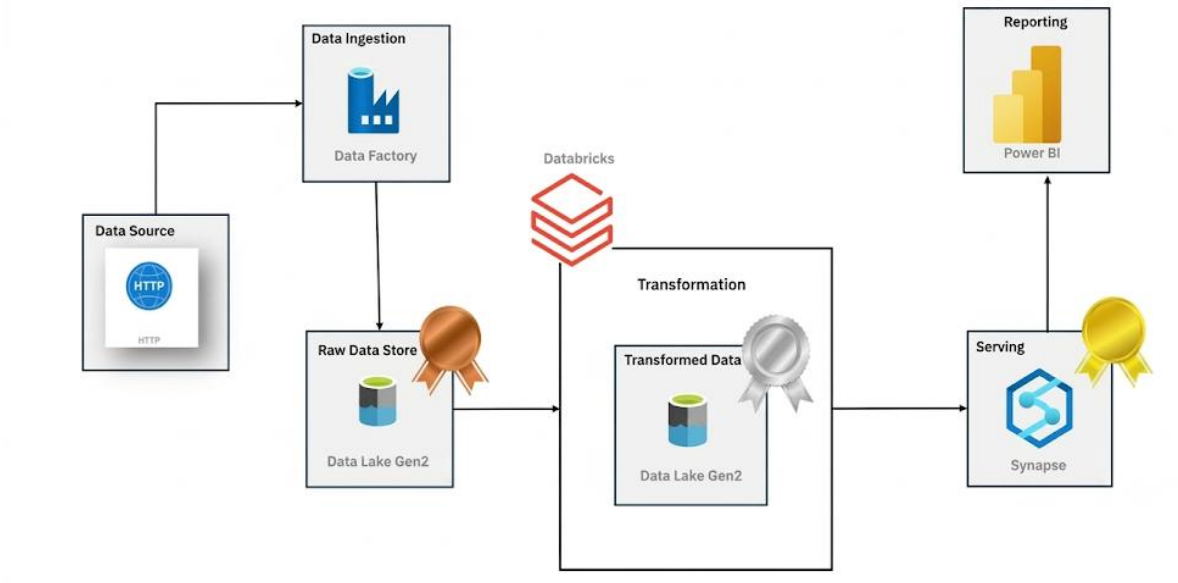
1. Project Overview

This project demonstrates an end-to-end Azure Data Engineering solution using the Medallion Architecture to ingest, transform, and serve AdventureWorks data for analytics. The pipeline centralizes raw operational data into a scalable cloud-based data lake, applies transformations using PySpark, and delivers an optimized analytical model for reporting through Azure Synapse Analytics and Power BI.

Dataset: Adventure Works (https://www.kaggle.com/datasets/ukveteran/adventure-works)

GitHub Repository: https://github.com/summyyyyy/Adventure-Work-DE-Project.git

2. Solution Architecture



The architecture follows an ELT (Extract, Load, Transform) pattern:

Source: AdventureWorks CSV files hosted in a GitHub repository.

Ingestion: ‘Azure Data Factory (ADF)’ orchestrates dynamic pipelines to land raw data in ‘ADLS Gen2 (Bronze Layer)’.

Processing: ‘Azure Databricks’ performs data transformation using ‘PySpark’ and stores the result in ADLS Gen2 (Silver Layer).

Warehousing: ‘Azure Synapse Analytics’ creates a serverless SQL pool to query the data, applying ‘CETAS’ for high-performance storage in ‘Parquet format (Gold Layer)’.

Serving: ‘ Power BI’ connects to the ‘Synapse SQL endpoint’ to visualize trends.

3. Technology Stack

Layer	Technology
Data Source	GitHub (AdventureWorks CSV files)
Orchestration	Azure Data Factory
Storage	Azure Data Lake Storage Gen2
Transformation	Azure Databricks (PySpark)
Data Warehouse	Azure Synapse Analytics (Serverless SQL)
Analytics	Power BI

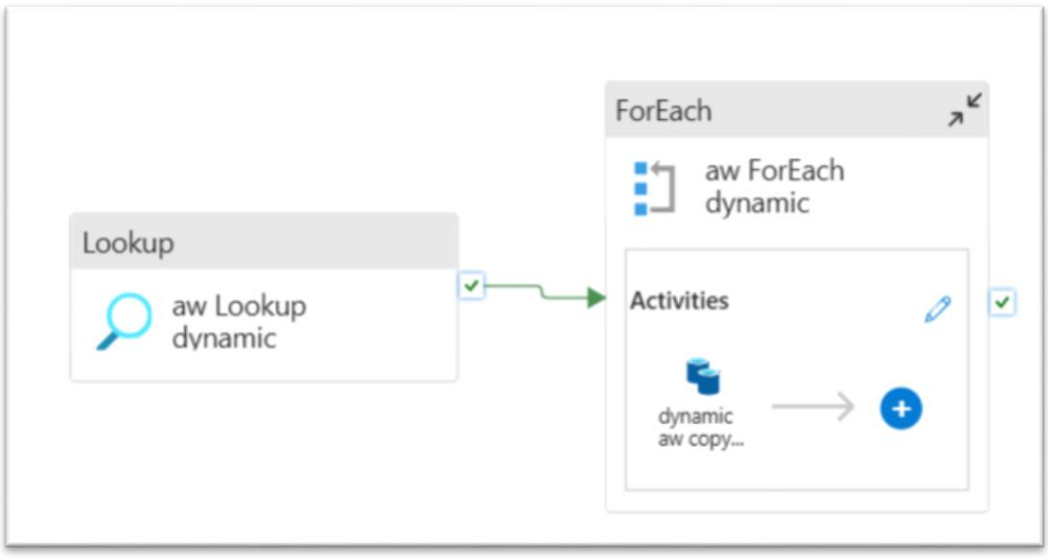
4. Project Phases

Phase 1: Data Ingestion (Bronze Layer)

The ingestion process preserves the source schema and stores the files without modification, ensuring data lineage and enabling future reprocessing if required.

Objective: Move data from an API source to the cloud storage while maintaining original formatting.

Implementation: ADF uses a `Lookup` activity to read a JSON-based configuration file, followed by a `ForEach` activity to dynamically run `Copy` activities for multiple CSV files.



Concepts: 'Dynamic Pipelines' allow for scalability, preventing the need for hardcoded pipelines for every new data source.

Note: When creating dynamic pipeline parameters, the components that vary for each file, such as the destination folder, file name, and source URL, are identified to enable parameterization.

Phase 2: Data Transformation (Silver Layer)

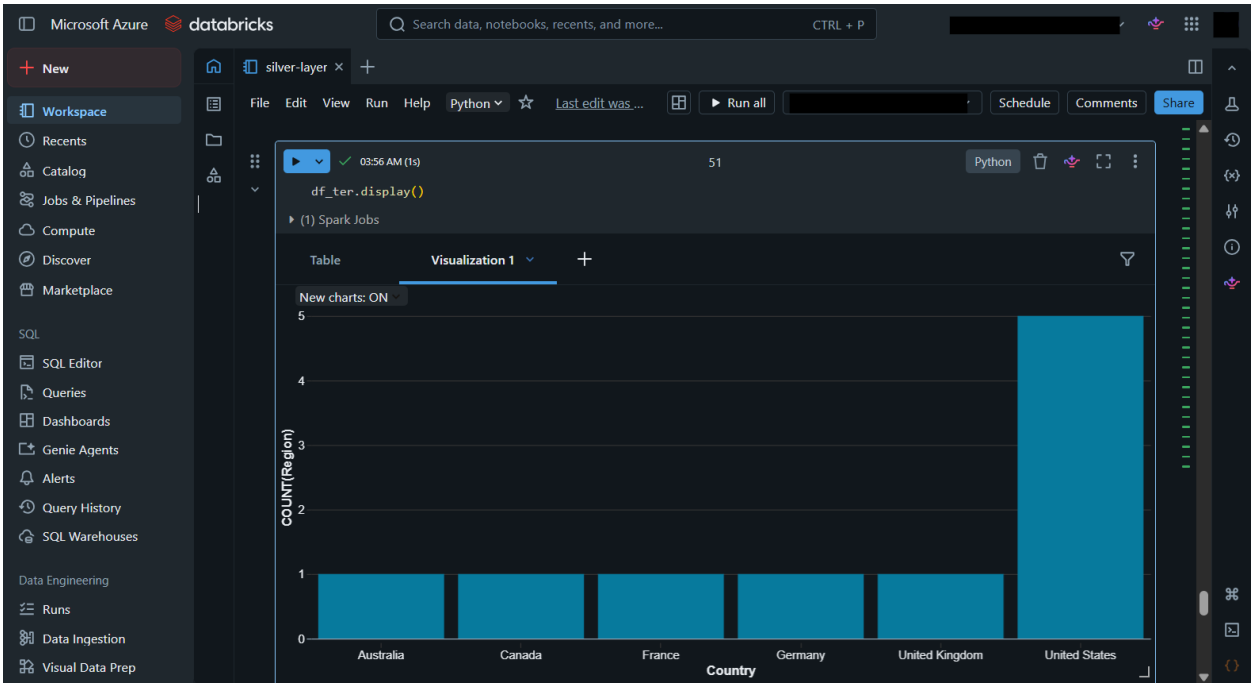
Objective: Clean, validate, and normalize the raw data.

Implementation: 'Azure Databricks' clusters are deployed. Using PySpark, the data is read from the Bronze layer, transformed (schema enforcement, filtering), and written to the Silver container as delta-ready files (parquet format).

Concepts: 'Service Principal' authentication is configured to securely grant Databricks access to the Data Lake.

Transformation notebook link: https://github.com/summyyyyy/Adventure-Work-DE-Project/blob/main/silver-layer.ipynb

Regional Coverage by Country using Databricks visualization:



Phase 3: Data Warehousing (Gold Layer)

Objective: Create a high-performance, analytical serving layer.

Implementation: Azure Synapse Serverless SQL was used to query the Silver layer using OPENROWSET(). CETAS (Create External Table As Select) was then used to materialize optimized Parquet datasets in the Gold layer, improving query performance and reducing repeated computation during BI reporting.

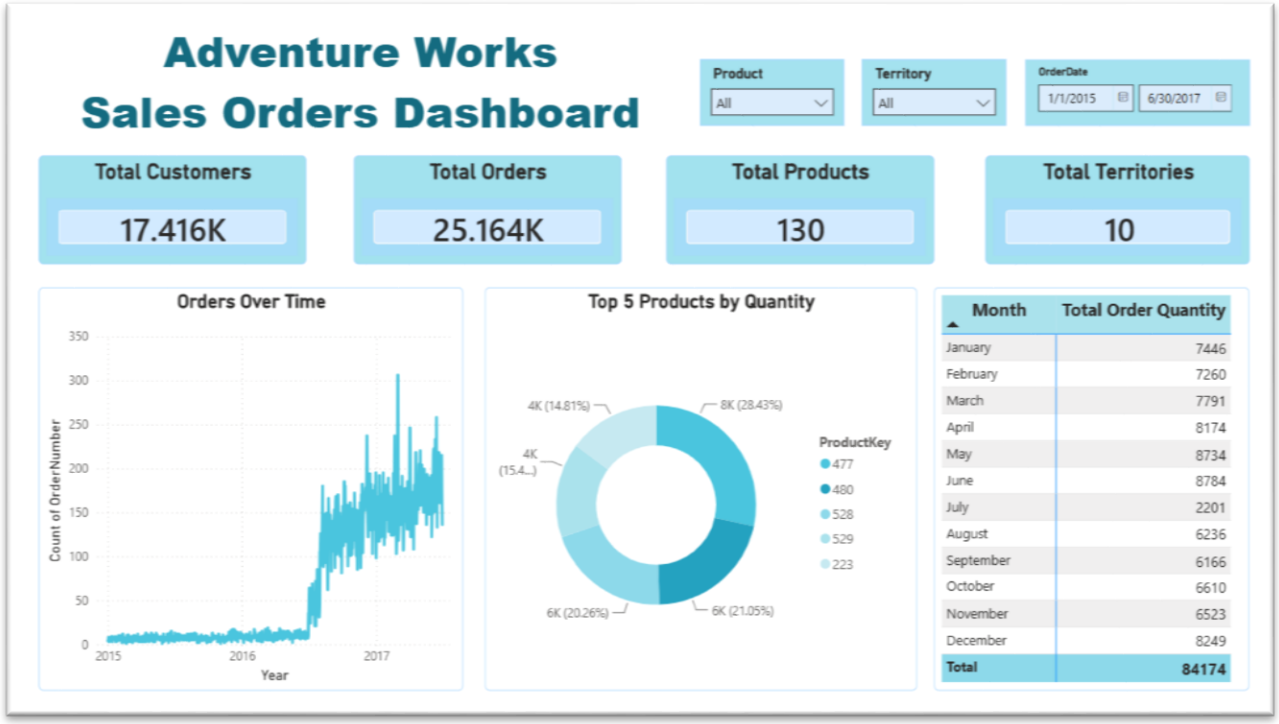
5. Key SQL and PySpark Logic

PySpark: `spark.read.format("csv").option("header", "true").load()` is the core method for initial ingestion. DataFrames are then cleaned using standard PySpark transformations.

SQL: The project heavily utilizes `CREATE EXTERNAL TABLE` and `OPENROWSET()` to provide a T-SQL interface over data lake files, effectively bridging the gap between a Data Lake and a Data Warehouse.

6. Reporting

Power BI Integration: The 'Synapse Serverless SQL endpoint' serves as the data source. Power BI authenticates using database-level credentials (created during the Synapse workspace setup) to pull data directly into the reporting model.



AdventureWorks Sales Orders Dashboard

Interactive Filters (Slicers)

Users can dynamically filter dashboard insights using:

- Order Date – Analyze sales trends across different time periods.
- Territory – Compare performance across different sales regions.
- Product – Explore product-level sales performance.

Key Performance Indicators (KPI Cards)

The dashboard displays key business metrics:

- Total Orders

  COUNT(OrderNumber) → Shows the total number of sales orders.

- Total Customers

  DISTINCTCOUNT(CustomerKey) → Displays the number of unique customers.
- Total Products

  DISTINCTCOUNT(ProductKey) → Represents the number of unique products sold.
- Total Territories

  DISTINCTCOUNT(TerritoryKey) → Shows the number of active sales territories.

Visualizations

1. Orders Over Time (Line Chart)

- Displays order volume trends across time.
- Helps identify seasonal patterns and changes in sales activity.

2. Top 5 Products by Quantity Sold (Donut Chart)

- Highlights the best-performing products based on total quantity ordered.
- Provides insights into product demand and contribution to overall sales.

3. Monthly Order Quantity Analysis (Matrix Visual)

- Shows order quantities summarized by month.
- Enables comparison of monthly sales performance and identifies high-performing periods.

7. Best Practices & Learnings

Data Quality: Schema validation was performed before writing data into the Silver layer to ensure consistency.

Storage Optimization: Parquet format was used because it provides efficient compression and columnar storage for analytical workloads.

Reusability: ADF pipelines were parameterized using metadata-driven configurations, allowing new datasets to be ingested without modifying pipeline logic.

Medallion Architecture: The separation into Bronze (raw), Silver (cleaned), and Gold (refined) is crucial for data governance.

Security: Using ‘Managed Identities’ is preferred over hardcoded account keys to maintain secure access between Azure services.

8. Github Repository Structure

```
AdventureWorks-DE-Project

├── data/
│   └── AdventureWorks dataset files

├── azure-databricks/
│   └── silver-layer.ipynb
│       └── For PySpark transformations for Silver layer processing

├── azure-synapse/
│   ├── Create External Table.sql
│   │   └── For Creates external tables using Synapse Serverless SQL

│   └── Create Views Gold.sql
│       └── For Creates Gold layer views for analytical reporting
```